

Definition

Ein Verteiltes System ist eine Kollektion unabhängiger Computer, die den Benutzern als ein Einzelcomputer erscheinen.

Ein Verteiltes System ist ein System in dem

- Hard- und Softwarekomponenten
- die sich auf miteinander vernetzten Computern befinden
- miteinander kommunizieren und ihre Aktionen koordinieren
- indem sie Nachrichten austauschen
- besteht aus einer Menge autonomer Computer (auch Komponenten)
- die durch ein Computernetzwerk (Kommunikationsbus) miteinander verbunden sind
- und mit einer Software zur Koordination ausgestattet sind
- einheitliches Gesamtsystem
- Ressourcen wie Hardware, Software & Daten

Eine **verteilte Anwendung** ist eine Anwendung, die ein Verteiltes System zur Lösung eines Anwendungsproblems nutzt. Sie besteht aus verschiedenen Softwarekomponenten, die mit den Komponenten des VS sowie den Anwendern kommuniziert.

Anwendung

- Per definition verteilte Anwendungen z.B. Messengerapplikationen
- bessere Verfügbarkeit und Fehlertoleranz
 - Redundanz von Hard- und Software
- Performance und Leistung
 - Rechenleistungsverteilung nach Anwendung
 - Verteilung über viele Rechner
- Lösen von zu großen Problemen wie KI Training und BigData

Probleme

- Kommunikationsversagen z.B. ein Knoten ist ausgefallen
- Rechenprozesse können ohne Kenntnis ausfallen
- Nicht-deterministische Fehler
- Probleme bei Entwicklung
 - Parallele Aktivitäten
 - globale Zeit schwer erfassbar

- kein konsistente Sicht des Gesamtzustandes
- zeit und räumliche Trennung von Ursache und Wirkung
- Räumliche Separation von Autonomen Komponenten
- Heterogenität
- Dynamik & Offenheit
- hohe Komplexität
- Sicherheitsgewährleistung
- Probleme bei sequenziellen Systemen
- Nebenläufigkeit
- Nichtdeterminismus
- Zustandsverteilung
- Schwere Synchronisation
- komplexe Programmierung
- aufwendiges Testen

8 falsche Annahmen zu Verteilten Systemen

- Netzwerk ist zuverlässig
- Latenz ist vernachlässigbar
- unbegrenzte Bandbreite
- Netzwerk ist sicher
- Topologie ändert sich nicht
- Es gibt nur einen Admin
- Netzwerkübertragung ist kostenlos
- Netzwerk ist homogen

Latenz

| Bearbeitungszeit von Router, Switch etc.

- hohe Latenz bei topologischer Trennung
- Örtlichkeit des Clients
- [Content Delivery Network](#)

Synchrone Verteilte Systeme

| System mit einer Menge von Prozessoren, die über einen internen Kommunikationsbus miteinander Verknüpft sind, und eine gemeinsame Uhr haben z.B. Supercomputer oder Mehrprozessorsystem

- Übertragung der Nachrichten brauchen nur begrenzte Zeit
- Beschränkte Zeitliche Verschiebung der Uhren
- $lb < ptime < ub$

- ptime = Ausführungszeit

Asynchrone Verteilte Systeme

- Latenz ist beliebig hoch
- zeitliche Verschiebung der Systemuhren ist beliebig hoch
- Bearbeitungsvorgang kann beliebig lange dauern
- Protokolle für asynchrone Systeme funktionieren auch für synchrone, aber nicht andersrum

Beispiele

- verteilte Internetanwendungen
- Ad-Hoc Netzwerkanwendungen
- Sensornetzwerke
- IoT

Partielle Synchrone Verteilte Systeme

- asynchrone Systeme mit lokalen Uhren, deren Zeit nicht beliebig auseinander driftet
- Latenz und Verarbeitung nur in Ausnahmefällen beliebig hoch
 - client und serverseitiger Timeout
- Modell für reale verteilte Systeme

Zentrale Begriffe und Eigenschaften

Richtige Verarbeitung in **endlicher** Zeit

- **Korrektheit**
 - Im Sinne der Applikationssemantik in jeder Bearbeitungssituation "richtiges Verhalten"
- **Lebendigkeit**
 - zeitnahe Ausführung von notwendigen Aktionen durch das System
- Ohne ist die Umsetzung eines verteilten Systems nicht möglich

QoS

- Performanz
- Verfügbarkeit
- Sicherheit
- Skalierbarkeit
- Zuverlässigkeit

Probleme

- Nebenläufigkeit
 - Notwendig für Skalierbarkeit und Fehlertoleranz
 - Verbesserung des Durchsatzes
- Koordinierung gemeinsamer Ressourcen
 - ohne = Verletzung der Korrektheit
 - falsch = Verletzung der Lebendigkeit
- Sicherheit
 - Vertraulichkeit
 - Integrität
 - Authentizität
 - Verfügbarkeit
 - Sicherheit für mobilen Code
- Transparenz bzw Blackbox Eigenschaften
 - Verbergung der Separation der einzelnen Komponente, sodass das System als ein Ganzes wahrgenommen wird und nicht als Sammlung von Komponenten
 - Zugriffstransparenz: Zugriff auf lokale und entfernte Ressourcen unter Verwendung identischen Operationen
 - Positions-/Ortstransparenz: Zugriff auf Ressourcen ohne Kenntnis über ihre Lokalität
 - Nebenläufigkeitstransparenz: gleichzeitiger Zugriff auf gemeinsame Ressourcen durch mehrere Prozesse, ohne gegenseitige Störung
 - Replikationstransparenz: mehrere Instanzen einer Ressource verwenden, ohne dass Replikas bekannt sind
 - Fehlertransparenz: Verbergen von Fehlern und ausgefallenen Ressourcen
 - Mobilitätstransparenz: Verschiebung von Ressourcen ohne Beeinträchtigung anderer Nutzer oder Programmen
 - Leistungstransparenz: Loadbalancing/ Leistungsanpassung bei variabler Last
 - Skalierungstransparenz: System und Applikation können skalieren, ohne die Systemstruktur und Algorithmen anzupassen
- Offenheit/Interoperabilität durch Schnittstellen

Systemmodelle

Beschreibung allgemeiner Eigenschaften des Designs eines Systems

- Deckt ab:
 - Systemkomponenten
 - Interaktion dieser
 - Beeinflussungsmöglichkeiten von kollektivem und individuellem Verhalten
- Beschreibt
 - Vereinfachung und Abstrahierung der Systemkomponenten

- Verteilung dieser im Netzwerk
- Beziehung zueinander

Softwarearchitektur

Strukturen des Systems aus Komponenten mit extern sichtbaren Eigenschaften dieser und deren Beziehungen

Komponenten und Schichten ausgespaart weil the fuck

Schichtenmodelle

1-Schicht

- Mainframe typisch. Alles zentral auf einem Rechner

2-Sicht:

- Klassisch Client/Server
 - Präsentation | alles andere
 - 2 Clients 1 Server
- Proxyserver: Gemeinsamer Cache für erhöhte Leistung und Verfügbarkeit
- Gleichrangige Prozesse: Jede Applikation koordiniert mit jeder

3-Ebenen

- Präsentation
- Businesslogik
- Daten

4-5 Ebenen

- Präsentation
 - Dialogsteuerung -> Koordinierung mit Anwender und Applikation
 - Anwendungslogik
 - Dienste und Datenlogik
 - Datenzugriff
 -
4. Schicht oftmals Weblayer

Middleware

Kommunikationsbrücke zwischen einzelnen Schichten

- Trennung von Anwendungs und Kommunikationslogik
- Interoperabilität über Rechnergrenzen hinaus

- Abstrahierung der Hardware
- Bereitstellung von Hilfsdiensten

Microservices

Funktionale Dekomposition in isolierte und einzeln wartbare deploybare Einheiten

- Skalierung über Größe des Entwicklungsteams (meistens 4-9 Leute)
- Vertikale Zerlegung in Funktionen über Schichten hinweg, nicht horizontal
- Kein gemeinsamer Zustand möglich

